
A07

REST API Client Reading the Docs

Algorithmic Thinking
ALD (MTD.ba VZ)
SS26

Submission via Moodle

24 Points

In the exercise you built a full client/server REST application. For this homework you only build the **client half**, but against a **real, public REST API of your own choice**. The point of this assignment is a skill you will need constantly as a professional: **read unfamiliar API documentation, figure out the JSON it returns, model it in Java, and consume it robustly**.

This is an **individual** assignment.

Submission Requirements:

- Read the [Organizational Guidelines](#) on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and explanations
 - Include all source files in a ZIP archive
 - Use the naming convention: `ALD_A07_LASTNAME.pdf` and `ALD_A07_LASTNAME.zip`
-

API Selection Rules

You may use **any** REST API you like — pick one whose topic you find interesting (animals, weather, transit, games, space, ...). A good catalogue of free APIs is <https://github.com/public-apis/public-apis>. Your chosen API *must* satisfy all of the following:

1. **Public and no authentication.** No API key, no sign-up, no OAuth. (A key that must be requested or paid for is *not* allowed.)
2. **REST returning JSON over HTTPS.** Not GraphQL, not SOAP, not HTML scraping.
3. **Non-trivial response.** The JSON must contain **nested objects and/or arrays**, so that you model more than a single flat class.
4. **Reproducible.** If you do *not* use TheCatAPI or the Dog CEO API, you **must commit one saved sample JSON response** (a real response, saved to a file) so your tests and grading work even if the API is offline or rate-limited.

Two documented examples (both public, both need no key):

- **TheCatAPI** — rich, nested data: <https://api.thecatapi.com/v1/breeds>
 - **Dog CEO API** — simple, flat data: <https://dog.ceo/dog-api/documentation/>
-

Exercise 1 (12 points) Exercise 1 — Reading the Docs and Consuming the API

Part A — Choose and Document an API (2 points)

Pick an API that satisfies the **API Selection Rules** above. In your PDF, document:

- which API and which endpoint(s) you use, and the base URL;
- one example request and (an excerpt of) its JSON response;
- which fields you decided to model and which you ignore.

Part B — Model the Response as DTOs (4 points)

Create plain Java DTO classes that mirror the JSON.

- At least one DTO must contain a **nested object or a list** (model the nested JSON object as its own class).
- Use a JSON library (Jackson or Gson). Configure it to **ignore unknown fields** so the real API's extra fields do not break parsing.
- Map JSON keys that are not valid Java field names (e.g. `snake_case` like `life_span`) explicitly.

Part C — The API Client (6 points)

Write a client class that uses `java.net.http.HttpClient` (no Spring) and your JSON library:

- a method that fetches a **collection** and returns a `List<YourDto>`;
- a method that fetches a **single resource** and returns `Optional<YourDto>`, returning `Optional.empty()` when the resource does not exist (HTTP 404);
- set a connection/request timeout.

Exercise 2 (12 points) Exercise 2 — Robustness, Tests, and Demo

Part A — A Second Operation (4 points)

Add one more meaningful operation, for example:

- a **search or filtered** query (e.g. `/search?q=...`), or
- a second, different endpoint of the same API.

Part B — Error Handling (3 points)

Your client must not crash on bad input or a bad network. Handle at least:

- a **non-200** status code (throw a clear exception, do not silently return wrong data);
- a **timeout** / connection error;
- an **empty result** (e.g. a search with no matches → empty list, not a crash).

Part C — Test with a Saved Sample (3 points)

Write JUnit 5 tests that parse a **saved sample JSON response** (a real response stored in `src/test/resources/`) into your DTOs and assert the mapping — including the **nested** field. Testing against a saved file (instead of the live API) keeps the test **deterministic and offline**; a flaky test that needs the internet is a bad test. **If your API is not Cat/Dog, committing this sample file is mandatory.** Show `mvn test` output in your PDF.

Part D — Console Demo (2 points)

A small Scanner menu that exercises your operations end-to-end against the live API. Include console output in your PDF.

Bonus (up to 3 points) — Asynchronous Requests.

Fetch several resources **concurrently** with `HttpClient.sendAsync` and `CompletableFuture.allOf`, instead of one blocking call after another. Show that the total time is close to the slowest single request rather than the sum.

Hints

- **Start by exploring with a browser or curl.** Look at the raw JSON *before* writing any Java — that is how you decide your DTO shape.
- **HttpClient basics:** `HttpClient.newHttpClient()`, build an `HttpRequest` with `.uri(...)`, send with `HttpResponse.BodyHandlers.ofString()`, read `response.statusCode()` and `response.body()`.
- **Jackson:** `new ObjectMapper().readValue(json, MyDto.class)`; for a list use a `TypeReference<List<MyDto>> {}`.
- **Save a sample response** for tests with `curl URL > src/test/resources/sample.json`.